InfraPilot: An Explainable Agentic Network Operations Center for AI-Assisted Infrastructure Diagnosis

Rohit GP[1], Patrick Alex Emmanuel V [2]
Department of Information Technology, Artificial Intelligence and Machine Learning
Saveetha Engineering College
Chennai, Tamil Nadu, India
{gp.rohit1201, patrickvijayanand2006}@gmail.com

***Abstract*—Modern IT infrastructures comprise interconnected networks, servers, databases, and applications that require continuous monitoring to ensure service availability and operational reliability. Although conventional monitoring platforms effectively detect failures and generate alerts, identifying the underlying cause of an incident remains a manual, time-consuming process that often requires administrators to correlate diagnostic evidence from multiple tools.**

This paper presents InfraPilot, an explainable Agentic Network Operations Center (NOC) prototype that integrates automated infrastructure diagnostics with collaborative multi-agent artificial intelligence for evidence-driven incident analysis. The proposed system combines a FastAPI orchestration service, a Python-based diagnostics engine, Redis Streams, LangGraph-based reasoning agents, and a React dashboard to automate the complete diagnostic workflow.

Infrastructure evidence collected from network, application, and system-level probes is analyzed by specialized AI agents responsible for network, database, and infrastructure reasoning. A supervisor agent consolidates the findings into ranked diagnostic hypotheses supported by evidence-based confidence scores and transparent reasoning traces. Furthermore, a human-in-the-loop approval mechanism ensures that operational decisions remain under administrator control.

Unlike conventional AI-assisted monitoring systems that produce opaque recommendations, InfraPilot emphasizes explainability by explicitly linking every diagnosis to the supporting infrastructure evidence. The proposed architecture demonstrates how distributed systems, agentic artificial intelligence, and explainable reasoning can be integrated into a lightweight, reproducible implementation of a modern Network Operations Center suitable for academic research and education.

***Index Terms*—Agentic Artificial Intelligence, Network Operations Center, Explainable Artificial Intelligence, Infrastructure Diagnostics, LangGraph, Redis Streams, Root Cause Analysis, Distributed Systems.**

I. Introduction

The rapid evolution of cloud computing, distributed systems, virtualization, and containerized applications has significantly increased the complexity of modern IT infrastructures. Organizations rely on uninterrupted availability of servers, databases, applications, and network services to support critical business operations. Consequently, Network Operations Centers (NOCs) play a vital role in continuously monitoring infrastructure health, detecting failures, and coordinating incident response activities [1]–[4].

Traditional monitoring platforms such as Nagios, Zabbix, Prometheus, and Grafana efficiently collect operational metrics and generate alerts when abnormal conditions are detected. However, identifying the actual root cause of an infrastructure failure remains largely dependent on experienced administrators. Operators typically execute multiple diagnostic utilities, including network reachability tests, DNS lookups, TCP port checks, service status verification, resource utilization analysis, and log inspection before manually correlating the collected evidence to determine the source of the problem. As infrastructure environments continue to grow in scale and complexity, this manual troubleshooting process becomes increasingly time-consuming and susceptible to human error [1]–[4].

Recent advances in Large Language Models (LLMs) and agentic artificial intelligence have introduced new possibilities for intelligent infrastructure management. Instead of merely summarizing alerts, AI agents can reason over structured diagnostic evidence, evaluate multiple potential causes, and generate human-readable explanations. Nevertheless, many existing AI-assisted monitoring solutions function as black-box systems that provide conclusions without exposing the reasoning process or the evidence supporting their decisions. Such limited transparency reduces administrator trust and complicates the validation of AI-generated recommendations [5]–[7].

Explainable Artificial Intelligence (XAI) addresses this limitation by enabling AI systems to justify their conclusions using interpretable reasoning. Within infrastructure operations, explainability requires every diagnosis to be directly supported by measurable observations such as network latency, DNS responses, port availability, CPU utilization, memory usage, service health, and application logs. Transparent reasoning improves operator confidence, facilitates auditing, and supports more informed operational decision-making [8], [9].

To address these challenges, this paper proposes **InfraPilot**, an explainable Agentic Network Operations Center that com-

bines automated infrastructure diagnostics with collaborative multi-agent reasoning. The proposed architecture integrates a FastAPI orchestration service, a Python-based diagnostics engine, Redis Streams for asynchronous evidence communication, LangGraph-based reasoning agents, and a React dashboard to automate the complete diagnosis workflow. Specialized AI agents independently analyze different categories of infrastructure evidence before a Supervisor Agent consolidates their findings into ranked diagnostic hypotheses accompanied by evidence-based confidence scores and transparent reasoning traces [10]–[13].

Unlike enterprise-grade commercial monitoring platforms, InfraPilot is intended as a lightweight, reproducible, and educational implementation that demonstrates the architectural principles of modern explainable Agentic NOCs. By separating evidence collection from AI reasoning through an event-driven architecture, the proposed system promotes modularity, scalability, and maintainability while preserving human oversight through a human-in-the-loop approval mechanism.

The primary contributions of this paper are summarized as follows:

- An explainable Agentic Network Operations Center architecture integrating automated infrastructure diagnostics with collaborative AI reasoning.
- A modular diagnostics framework for automated network, service, and system health monitoring.
- An evidence-driven reasoning mechanism that generates multiple ranked diagnostic hypotheses supported by measurable confidence scores.
- A distributed event-driven communication model using Redis Streams to decouple infrastructure diagnostics from AI reasoning.
- A human-in-the-loop workflow that preserves administrator oversight while providing AI-assisted diagnostic recommendations.

The remainder of this paper is organized as follows. Section II reviews related work on infrastructure monitoring, explainable AI, and multi-agent systems. Section III presents the proposed methodology and system architecture. Section IV describes the implementation and system workflow. Section V evaluates the proposed framework, and Section VI concludes the paper with directions for future work.

II. Related Work

Research on intelligent infrastructure operations spans several related areas, including automated root-cause analysis (RCA), Artificial Intelligence for IT Operations (AIOps), explainable fault diagnosis, multi-agent decision support, and event-driven monitoring. These areas provide the foundation for the design of InfraPilot.

A. Automated Root-Cause Analysis

Root-cause analysis in distributed and cloud-native systems has been extensively studied because failures can propagate across heterogeneous services and infrastructure components. CloudRanger constructs dynamic causal relationships between applications and uses graph-based analysis to identify services responsible for cloud incidents [14]. MicroRCA similarly addresses performance diagnosis in microservice environments by correlating application symptoms with system resource utilization and modeling anomaly propagation across services and machines [15].

Other approaches have explored failure diagnosis through fault injection and historical failure matching. Pham *et al.* use targeted fault injection to construct a database of failure behaviours and subsequently match production failures against previously observed patterns [16]. More recently, COCA incorporates code knowledge and execution-path information into LLM-based RCA for distributed-system issue reports, demonstrating the growing use of generative models for automated diagnosis [17].

These approaches demonstrate that effective RCA benefits from correlating heterogeneous evidence rather than relying on a single operational signal. However, many existing approaches are specialized for particular environments, require extensive telemetry or system-specific models, or focus primarily on automated localization rather than human-readable diagnostic explanations.

B. AIOps and Intelligent IT Operations

AIOps combines artificial intelligence with operational data to automate activities such as anomaly detection, incident analysis, root-cause identification, and remediation. A recent survey of AIOps research identifies failure perception, root-cause analysis, and automated remediation as major AIOps tasks and highlights the increasing use of Large Language Models (LLMs) for operational reasoning [18].

The emergence of LLM-based AIOps creates opportunities for more flexible diagnostic workflows because language models can reason over heterogeneous operational information. However, operational systems still require reliable evidence, structured workflows, and appropriate safeguards because incorrect AI-generated conclusions can lead to inappropriate operational decisions.

C. Explainable Fault Diagnosis

Explainability is particularly important for infrastructure diagnosis because administrators need to understand why a system identifies a particular component or condition as a potential cause. A systematic review of explainable AI for industrial fault diagnosis identifies interpretability, transparency, trust, and auditability as important requirements for deploying AI-based fault-diagnosis systems [19]. These requirements are also relevant to IT infrastructure, where diagnostic recommendations should be associated with observable operational evidence.

InfraPilot therefore emphasizes evidence-linked explanations rather than presenting the output of the language model as an unexplained prediction. Probe observations, probe confidence values, ranked hypotheses, and reasoning traces are retained as part of the diagnostic result.

D. Multi-Agent Decision Support

Multi-agent systems provide a natural framework for decomposing complex decision problems into specialized tasks. Rizk *et al.* survey cooperative decision making in multi-agent systems and identify coordination, scalability, and distributed decision making as important research challenges [20]. Vahidov and Fazlollahi propose a pluralistic multi-agent decision-support framework in which agents are organized around different phases of human problem solving, demonstrating the relevance of agent-based architectures to decision-support applications [21].

These principles motivate the separation of diagnostic responsibilities in InfraPilot. Specialized agents analyze different evidence domains, while a Supervisor Agent combines their findings into a unified diagnostic report. This structure supports modular reasoning while retaining human oversight over operational decisions.

E. Event-Driven Monitoring

Event-driven processing has also been investigated for monitoring distributed and networked infrastructures. Frankowski *et al.* apply Complex Event Processing to network monitoring and anomaly detection, emphasizing the need to integrate and process information from multiple infrastructure sensors and systems [22]. Such approaches demonstrate the value of processing operational observations as streams of events rather than treating every observation as an isolated request.

InfraPilot applies this principle through Redis Streams, which separates infrastructure evidence collection from the subsequent AI reasoning stage. This event-driven design allows diagnostic probes and reasoning components to operate independently.

F. Research Gap

Existing research demonstrates substantial progress in automated RCA, AIOps, explainable diagnosis, multi-agent decision support, and event-driven monitoring. These research areas have each received considerable attention, but their integration into a lightweight infrastructure-diagnosis workflow that combines automated evidence collection, event-driven communication, multi-agent reasoning, explainability, and human validation is less emphasized in the specific educational and reproducible prototype context targeted by this work.

InfraPilot combines these principles into a single lightweight diagnostic framework. The proposed architecture integrates automated infrastructure probes, asynchronous evidence streaming, specialized multi-agent reasoning, evidence-based confidence estimation, explainable diagnostic reports, and human-in-the-loop validation. The objective is not to replace existing enterprise monitoring platforms, but to provide a reproducible architecture for studying evidence-driven AI-assisted infrastructure diagnosis.

The scholarly sources discussed in this section provide the research foundation for the identified challenges and architectural motivations. Official documentation for technologies such as React, FastAPI, Redis Streams, LangGraph, PostgreSQL, and Docker is used primarily as a technical reference for implementation details and technology integration rather than as evidence for the research claims presented in this work.

TABLE I
SUMMARY OF INFRAPILOT CAPABILITIES

Capability	**Conv.**	**AIOps**	**InfraPilot**
Monitoring	Core	Core	Implemented
Alerting	Rule-based	Rules + anomaly detection	Probes
RCA	Manual	Partial–advanced	Hypotheses
Evidence correlation	Basic	Automated	Structured
Explainability	Limited	Variable	Evidence-linked
Multi-agent reasoning	No	Emerging	Yes
Confidence scoring	Severity-based	Variable	Implemented
Human oversight	Varies	Varies	Approval
Event-driven flow	Optional	Common	Redis Streams
Storage	Varies	Common	PostgreSQL
Remediation	Manual	Some systems	Out of scope
Evaluation	Tool-specific	RCA/MTTR	Future work

The contribution of InfraPilot is not the introduction of a new Large Language Model, root-cause analysis algorithm, or multi-agent paradigm. Instead, the work focuses on the integration and implementation of established concepts into a lightweight, reproducible architecture for infrastructure diagnosis. The proposed system separates automated evidence collection from AI reasoning through Redis Streams, assigns diagnostic responsibilities to specialized agents, preserves the operational evidence supporting generated hypotheses, and maintains administrator oversight through a human-in-the-loop workflow. This combination provides a practical framework for studying evidence-driven and explainable AI-assisted infrastructure diagnosis.

Table I provides a qualitative comparison of representative conventional monitoring, AIOps, and the implemented InfraPilot prototype. The comparison is informed by prior research on AIOps, automated root-cause analysis, and multi-agent diagnostic systems [14], [15], [17], [18], [20]. It is not intended as an empirical benchmark. The InfraPilot entries indicate implemented prototype capabilities, while quantitative effectiveness remains subject to further evaluation.

III. PROPOSED METHODOLOGY

InfraPilot is an explainable Agentic Network Operations Center (NOC) that automates infrastructure diagnosis through evidence-driven reasoning and collaborative AI agents. The proposed framework integrates automated diagnostics, asynchronous messaging, multi-agent reasoning, and human-in-the-loop validation to generate transparent diagnostic reports supported by measurable infrastructure evidence.

A. System Architecture

The overall architecture consists of seven major components: (i) React Dashboard, (ii) FastAPI Orchestration Service, (iii) Python Diagnostics Engine, (iv) Redis Streams, (v) LangGraph Multi-Agent Engine, (vi) Cloud-hosted Large Language Model (LLM), and (vii) PostgreSQL Database.

Figure 1 presents the overall architecture.

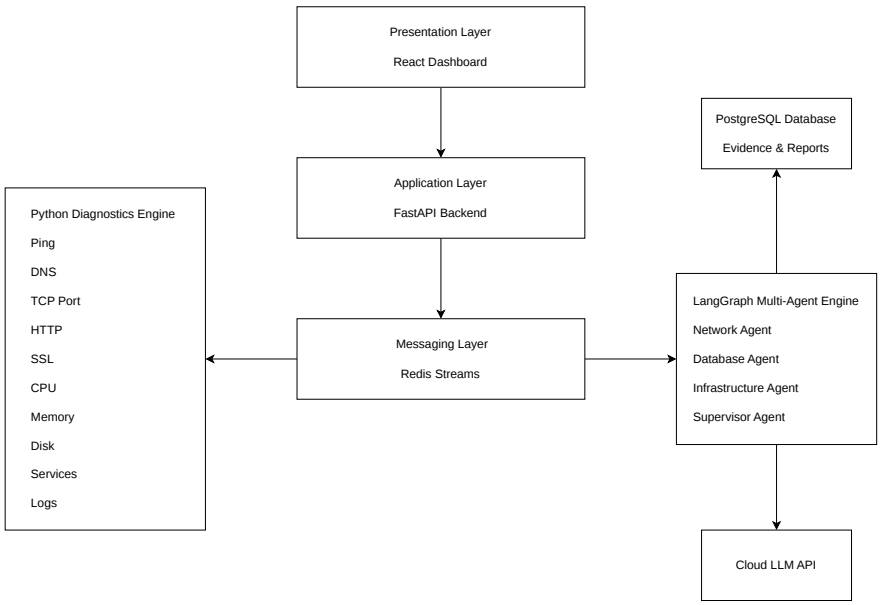

Fig. 1. Overall architecture of the proposed InfraPilot framework.

The React dashboard allows administrators to initiate diagnosis requests and visualize AI-generated reports. FastAPI orchestrates diagnosis jobs and exposes REST APIs, while the Python diagnostics engine executes infrastructure probes. Probe results are published asynchronously through Redis Streams and consumed by LangGraph, where specialized AI agents analyze the collected evidence using a cloud-hosted LLM. PostgreSQL stores probe results, reasoning traces, confidence scores, and historical diagnosis records [10]–[13], [23].

B. Evidence Collection and Event Streaming

InfraPilot performs automated diagnostics using modular probes covering network connectivity, application availability, service health, and system resource utilization. Each probe generates a standardized evidence object represented as

$$E_i = (P_i, T_i, R_i, S_i, C_i, \tau_i) \tag{1}$$

where P_i denotes the probe type, T_i the target, R_i the observed result, S_i the probe status, C_i the probe confidence, and τ_i the execution timestamp.

Completed probe results are published to Redis Streams, forming the evidence stream

$$S = \{E_1, E_2, \ldots, E_n\}, \tag{2}$$

where n represents the number of executed probes. Redis Streams decouples diagnostics from AI reasoning, enabling asynchronous execution, persistent messaging, replay capability, and improved fault tolerance [11].

C. Multi-Agent Diagnostic Reasoning

After evidence becomes available in Redis Streams, LangGraph coordinates multiple specialized reasoning agents. Dedicated agents independently analyze different categories of infrastructure evidence, including network connectivity, system resources, application services, and operational logs. A Supervisor Agent aggregates their findings, invokes the cloud-hosted LLM when required, and generates ranked diagnostic hypotheses together with explainable reasoning [6], [7], [12].

The final hypothesis set is represented as

$$H = \{H_1, H_2, \ldots, H_m\}, \tag{3}$$

where each hypothesis corresponds to a potential root cause supported by the collected infrastructure evidence.

D. Probe Confidence Estimation

Each probe computes a confidence score based on its execution status and observed latency. The initial confidence is assigned according to the probe status:

$$C_b = \begin{cases} 100, & \text{OK} \\ 80, & \text{Degraded} \\ 60, & \text{Failed} \\ 30, & \text{Error.} \end{cases} \tag{4}$$

Latency penalties are then applied cumulatively (5 points for latency exceeding 100 ms, an additional 10 points beyond 300 ms, and a further 15 points beyond 1000 ms). The final confidence is calculated as

$$C = \max(0, C_b - P_L), \tag{5}$$

where P_L denotes the cumulative latency penalty.

The Supervisor Agent computes the overall diagnostic confidence as

$$C_{\text{overall}} = \frac{1}{n} \sum_{i=1}^{n} C_i, \tag{6}$$

where C_i is the confidence score of the i^{th} probe.

E. Human-in-the-Loop Validation

Rather than automatically executing remediation actions, InfraPilot presents administrators with the collected evidence, ranked hypotheses, confidence scores, explainable reasoning, and suggested remediation plans. Administrators review and approve the recommendations before corrective actions are performed, ensuring that AI functions as a decision-support system while maintaining human oversight.

IV. Implementation and System Workflow

InfraPilot is implemented as a modular, event-driven system in which the user interface, backend services, diagnostics engine, AI reasoning, messaging, and persistent storage operate as independent components. Communication between services is performed through REST APIs and Redis Streams, enabling asynchronous execution and loose coupling.

A. System Implementation

The implementation consists of the following software components:

- React Dashboard
- FastAPI Orchestration Service
- Python Diagnostics Engine
- Redis Streams
- LangGraph Multi-Agent Engine
- PostgreSQL Database
- Docker Compose

The React dashboard provides an interface for initiating diagnosis requests and visualizing probe evidence, AI reasoning, confidence scores, and ranked diagnostic hypotheses. FastAPI acts as the orchestration layer by creating diagnosis jobs, invoking infrastructure probes, coordinating Redis communication, and exposing REST APIs to the frontend [10], [13].

The diagnostics engine is implemented using asynchronous Python modules that execute infrastructure probes covering network connectivity, service availability, and system resource utilization. Each completed probe generates a structured JSON evidence object that is published to Redis Streams [11].

LangGraph consumes the streamed evidence and coordinates multiple specialized reasoning agents. These agents analyze different categories of infrastructure evidence before a Supervisor Agent aggregates their outputs into an explainable diagnostic report with an overall confidence score. Historical evidence, reasoning traces, and administrator approvals are stored in PostgreSQL [12], [23].

All services are deployed using Docker Compose, providing isolated runtime environments and reproducible deployment across development and production systems [24].

B. System Workflow

Figure 2 illustrates the end-to-end execution workflow.

The workflow consists of the following stages:

1) The administrator initiates a diagnosis request from the React dashboard.
2) FastAPI creates a diagnosis job and triggers the diagnostics engine.
3) Infrastructure probes execute asynchronously and publish evidence to Redis Streams.
4) LangGraph consumes the evidence and coordinates specialized reasoning agents.
5) The Supervisor Agent synthesizes the agent outputs into ranked diagnostic hypotheses with confidence scores and explainable reasoning.
6) The completed diagnosis is stored in PostgreSQL and returned to the dashboard for administrator review.

The modular implementation separates orchestration, diagnostics, reasoning, and persistence into independent services, improving scalability, maintainability, and extensibility while preserving human oversight throughout the diagnostic process.

V. Evaluation

The InfraPilot prototype was evaluated through functional validation of its end-to-end diagnostic workflow. The evaluation focused on verifying the correct integration of infrastructure probing, asynchronous evidence transmission, multi-agent reasoning, confidence estimation, and diagnostic report generation.

A. Evaluation Criteria

The implementation was evaluated according to the following criteria:

- Successful execution of infrastructure diagnostic probes.
- Successful transmission of probe evidence through Redis Streams.
- Correct consumption and processing of evidence by the LangGraph reasoning workflow.
- Generation of ranked diagnostic hypotheses with confidence scores.
- Presentation of evidence and diagnostic results through the React dashboard.

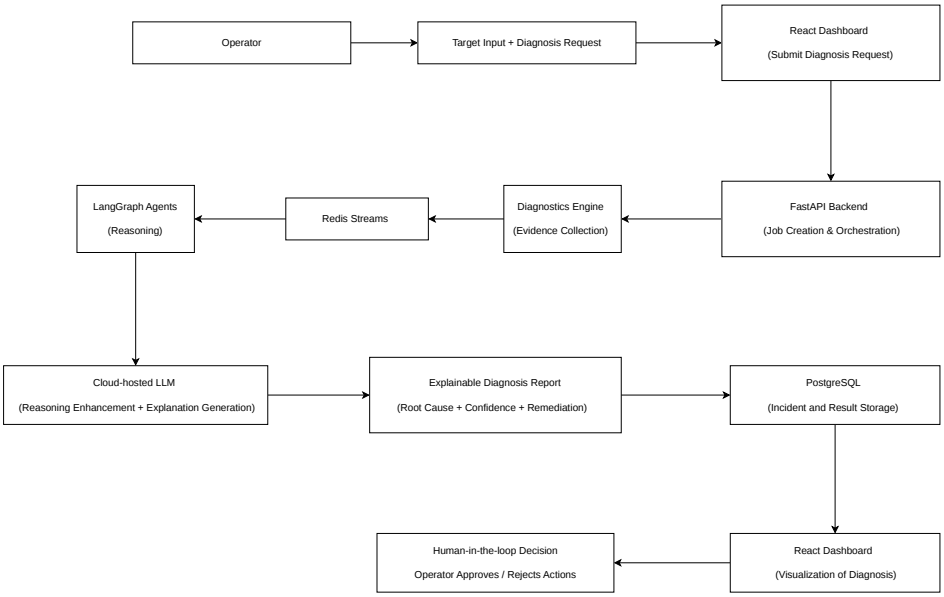

Fig. 2. End-to-end workflow of InfraPilot.

B. End-to-End Workflow Validation

A complete diagnosis request was used to validate the communication between the major system components. The request initiated from the React dashboard was received by the FastAPI orchestration service, which created the diagnosis job and triggered the Python diagnostics engine. The executed probes generated structured evidence containing probe status, observed results, latency, confidence score, and timestamp.

The generated evidence was successfully published to Redis Streams and consumed by the LangGraph reasoning workflow. Specialized agents analyzed the available evidence, after which the Supervisor Agent aggregated the intermediate findings and generated ranked diagnostic hypotheses together with an overall confidence score. The resulting diagnosis was then stored in PostgreSQL and made available to the React dashboard for administrator review.

This workflow validation confirms that the major components can operate as an integrated diagnostic pipeline while maintaining separation between evidence collection, asynchronous communication, AI reasoning, and persistent storage.

C. Discussion

The functional evaluation demonstrates the feasibility of the proposed architecture for evidence-driven infrastructure diagnosis. The diagnostic engine provides structured infrastructure observations, while Redis Streams decouples evidence collection from the subsequent reasoning stage. The LangGraph workflow further separates diagnostic responsibilities among specialized agents and provides a mechanism for synthesizing their findings into an explainable report.

The evaluation is primarily functional and does not constitute a quantitative comparison with existing monitoring or AIOps platforms. Future work will therefore include measurements of probe execution time, end-to-end response latency, throughput under concurrent diagnosis requests, reasoning accuracy, and comparison with conventional monitoring and root-cause analysis approaches.

VI. Conclusion

This paper presented InfraPilot, an explainable Agentic Network Operations Center (NOC) that integrates automated infrastructure diagnostics with collaborative AI reasoning to support intelligent incident analysis. The proposed framework combines a React-based user interface, FastAPI orchestration, a Python diagnostics engine, Redis Streams, LangGraph multi-agent coordination, cloud-hosted Large Language Models, and PostgreSQL to provide a modular and event-driven diagnostic platform.

Unlike conventional monitoring systems that primarily generate alerts, InfraPilot emphasizes evidence-driven diagnosis by combining structured infrastructure observations with explainable AI reasoning and confidence estimation. The proposed architecture separates evidence collection, asynchronous messaging, reasoning, and persistent storage into independent components, improving scalability, maintainability, and transparency while preserving human oversight through a human-in-the-loop approval workflow.

Future work will focus on extending the system with additional diagnostic agents, quantitative performance evaluation in large-scale network environments, automated remediation

recommendations, and integration with production monitoring platforms. These enhancements will further strengthen InfraPilot as a scalable and explainable AI-assisted solution for modern network operations.

REFERENCES

[1] *Nagios Documentation*, Nagios Enterprises, 2025, https://www.nagios.org/documentation/.

[2] *Prometheus Documentation*, Cloud Native Computing Foundation, 2025, https://prometheus.io/docs/.

[3] *Grafana Documentation*, Grafana Labs, 2025, https://grafana.com/docs/.

[4] *Zabbix Documentation*, Zabbix LLC, 2025, https://www.zabbix.com/documentation.

[5] A. Vaswani *et al.*, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[6] J. Wei *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[7] S. Yao *et al.*, "React: Synergizing reasoning and acting in language models," *International Conference on Learning Representations*, 2023.

[8] W. Samek, T. Wiegand, and K.-R. Müller, "Explainable artificial intelligence: Understanding, visualizing and interpreting deep learning models," *arXiv preprint arXiv:1708.08296*, 2017.

[9] R. Guidotti *et al.*, "A survey of methods for explaining black box models," *ACM Computing Surveys*, vol. 51, no. 5, pp. 1–42, 2018.

[10] *FastAPI Documentation*, FastAPI, 2025, https://fastapi.tiangolo.com/.

[11] *Redis Streams Documentation*, Redis, 2025, https://redis.io/docs/data-types/streams/.

[12] *LangGraph Documentation*, LangChain Inc., 2025, https://langchain-ai.github.io/langgraph/.

[13] *React Documentation*, Meta, 2025, https://react.dev/.

[14] P. Wang, J. Xu, M. Ma, W. Lin, D. Pan, Y. Wang, and P. Chen, "Cloudranger: Root cause identification for cloud native systems," in *2018 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*, 2018, pp. 492–502.

[15] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, "Microrca: Root cause localization of performance issues in microservices," in *2020 IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2020.

[16] C. Pham, L. Wang, B. C. Tak, S. Baset, C. Tang, Z. Kalbarczyk, and R. K. Iyer, "Failure diagnosis for distributed systems using targeted fault injection," *IEEE Transactions on Parallel and Distributed Systems*, vol. 28, no. 2, pp. 503–516, 2017.

[17] Y. Li, Y. Wu, J. Liu, Z. Jiang, Z. Chen, G. Yu, and M. R. Lyu, "Coca: Generative root cause analysis for distributed systems with code knowledge," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025, pp. 1346–1358.

[18] L. Zhang, T. Jia, M. Jia, Y. Wu, A. Liu, Y. Yang, Z. Wu, X. Hu, P. S. Yu, and Y. Li, "A survey of aiops in the era of large language models," *ACM Computing Surveys*, vol. 58, no. 2, pp. 1–35, 2026.

[19] J. Cação, J. S. Santos, and M. Antunes, "Explainable ai for industrial fault diagnosis: A systematic review," *Journal of Industrial Information Integration*, vol. 47, p. 100905, 2025.

[20] Y. Rizk, M. Awad, and E. W. Tunstel, "Decision making in multiagent systems: A survey," *IEEE Transactions on Cognitive and Developmental Systems*, vol. 10, no. 3, pp. 514–529, 2018.

[21] R. Vahidov and B. Fazlollahi, "Pluralistic multi-agent decision support system: A framework and an empirical test," *Information & Management*, vol. 41, no. 7, pp. 883–898, 2004.

[22] G. Frankowski, M. Jerzak, M. Miłostan, T. Nowak, and M. Pawłowski, "Application of the complex event processing system for anomaly detection and network monitoring," *Computer Science*, vol. 16, no. 4, p. 351, 2015.

[23] *PostgreSQL Documentation*, PostgreSQL Global Development Group, 2025, https://www.postgresql.org/docs/.

[24] *Docker Documentation*, Docker Inc., 2025, https://docs.docker.com/.